



University of the Punjab

(Gujranwala Campus)

ARTIFICIAL INTELLIGENCE

LAB TASK 01

Submitted by:

AIMAN DUHA

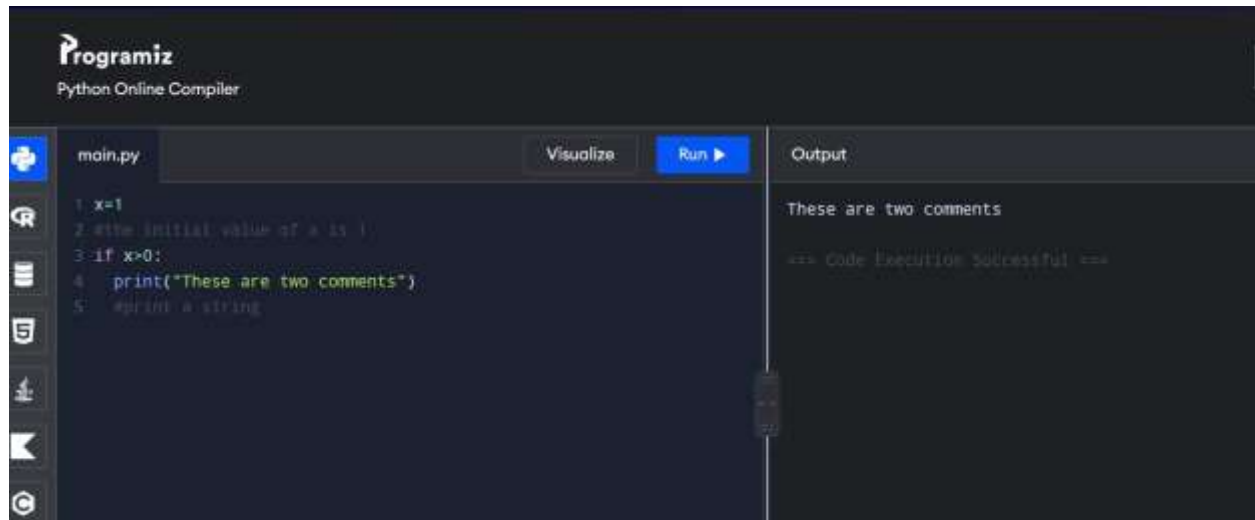
ROLL NO 33

BSIT (5th Semester)

Submitted to:

Sir Ghulam Mustafa

TASK # 1



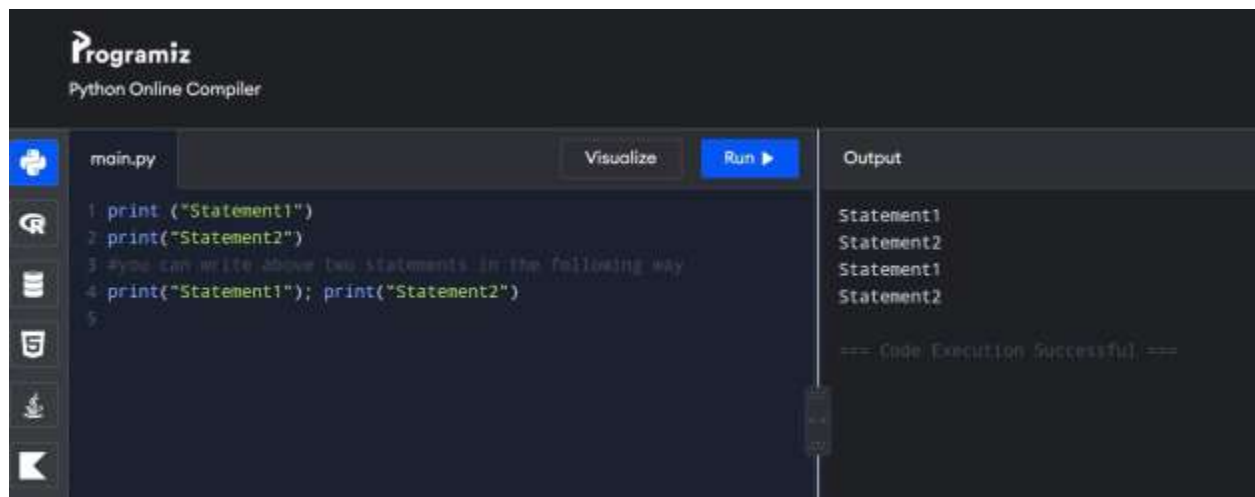
The screenshot shows the Programiz Python Online Compiler interface. The code editor on the left contains a Python script named 'main.py' with the following code:

```
1 x=1
2 #the initial value of a is 1
3 if x>0:
4     print("These are two comments")
5     #print a string
```

The 'Run' button is highlighted in blue. The output panel on the right displays the result of the execution:

```
These are two comments
=== Code Execution Successful ===
```

TASK # 2



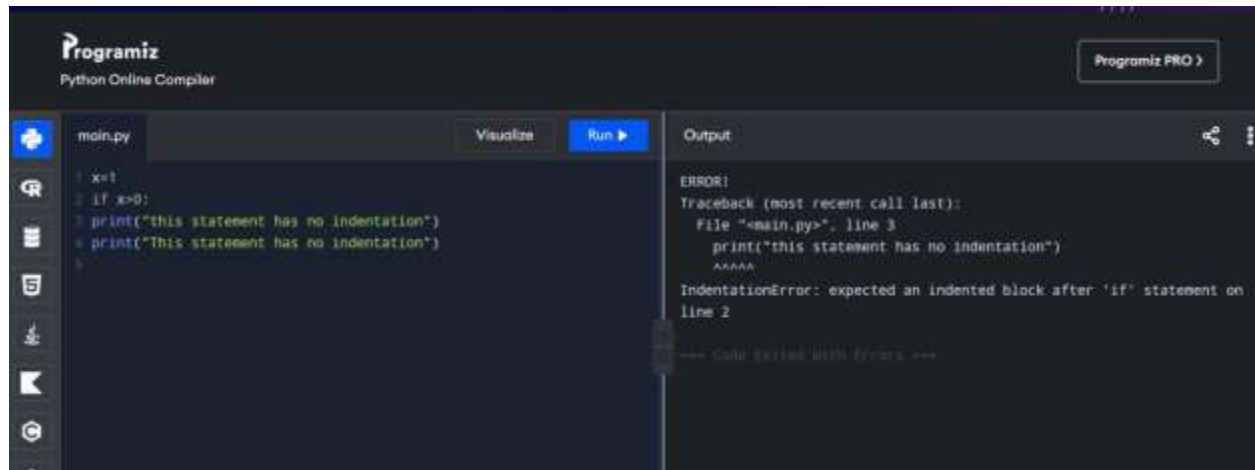
The screenshot shows the Programiz Python Online Compiler interface. The code editor on the left contains a Python script named 'main.py' with the following code:

```
1 print ("Statement1")
2 print("Statement2")
3 #you can write above two statements in the following way
4 print("Statement1"); print("Statement2")
5
```

The 'Run' button is highlighted in blue. The output panel on the right displays the result of the execution:

```
Statement1
Statement2
Statement1
Statement2
=== Code Execution Successful ===
```

TASK # 3



The screenshot shows the Programiz Python Online Compiler interface. The code editor on the left contains the following Python code:

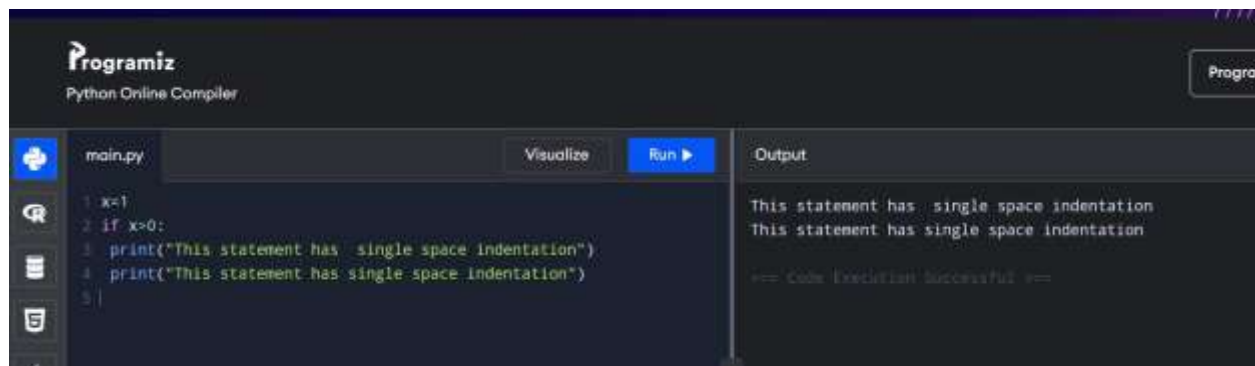
```
1 x=1
2 if x>0:
3 print("this statement has no indentation")
4 print("This statement has no indentation")
5
```

The code is syntactically incorrect due to inconsistent indentation. The output panel on the right displays the following error message:

```
ERROR!
Traceback (most recent call last):
  File "<main.py>", line 3
    print("this statement has no indentation")
    ^^^^^
IndentationError: expected an indented block after 'if' statement on line 2
```

Below the error message, it says "Code failed with errors".

TASK # 4



The screenshot shows the Programiz Python Online Compiler interface. The code editor on the left contains the following Python code:

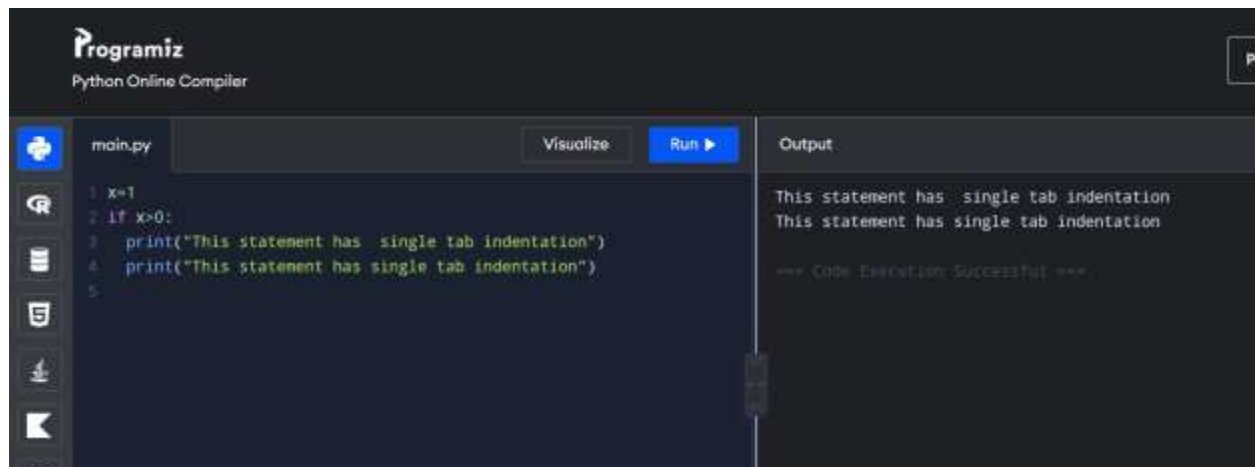
```
1 x=1
2 if x>0:
3 print("This statement has single space indentation")
4 print("This statement has single space indentation")
5
```

The code is syntactically correct. The output panel on the right displays the following output:

```
This statement has single space indentation
This statement has single space indentation
```

Below the output, it says "Code Execution Successful".

TASK # 5



The screenshot shows the Programiz Python Online Compiler interface. The code editor on the left contains the following Python code:

```
1 x=1
2 if x>0:
3 print("This statement has single tab indentation")
4 print("This statement has single tab indentation")
5
```

The code is syntactically correct. The output panel on the right displays the following output:

```
This statement has single tab indentation
This statement has single tab indentation
```

Below the output, it says "Code Execution Successful".

TASK # 6

The screenshot shows the Programiz Python Online Compiler interface. The editor contains the following Python code:

```
1 x=1
2 if x>0:
3     print("This statement has single tab+space indentation")
4     print("This statement has single tab+space indentation")
5
```

The output pane displays the result of the execution:

```
This statement has single tab+space indentation
This statement has single tab+space indentation

=== Code Execution Successful ===
```

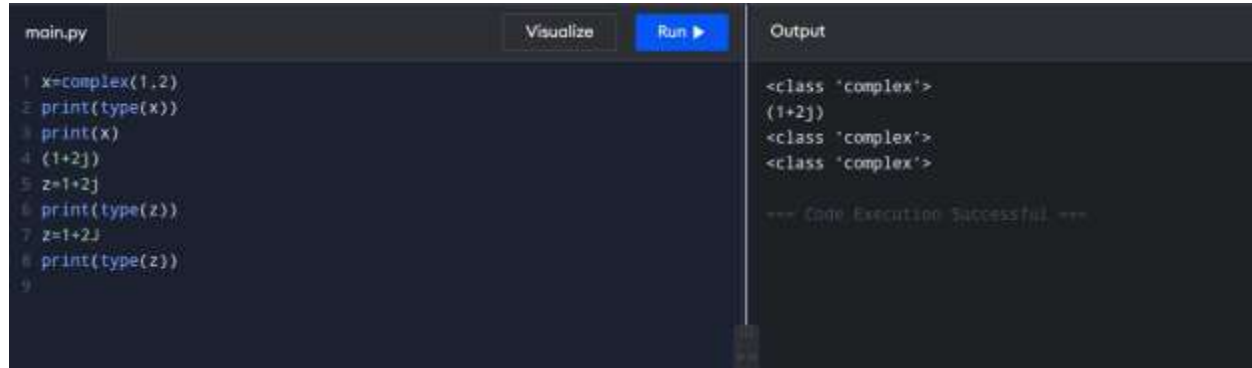
Task #7

```
main.py Visualize Run ▶ Output
```

```
1 a=123  
2 print(type(a))  
3  
4 b=-4587  
5 print(type(b))  
6  
7 c=0  
8 print(type(c))  
9  
10 g=1.03  
11 print (type(g))  
12  
13 h=-11.23  
14 print(type(h))  
15  
16 i=.34  
17 print(type(i))  
18  
19 j=2.12e-10  
20 print(type(j))  
21  
22 k=5E220  
23 print(type(k))  
24
```

```
<class 'int'>  
<class 'int'>  
<class 'int'>  
<class 'float'>  
<class 'float'>  
<class 'float'>  
<class 'float'>  
<class 'float'>  
  
=== Code Execution Successful! ===
```

TASK #8



The screenshot shows a Python IDE with a file named 'main.py'. The code in the editor is as follows:

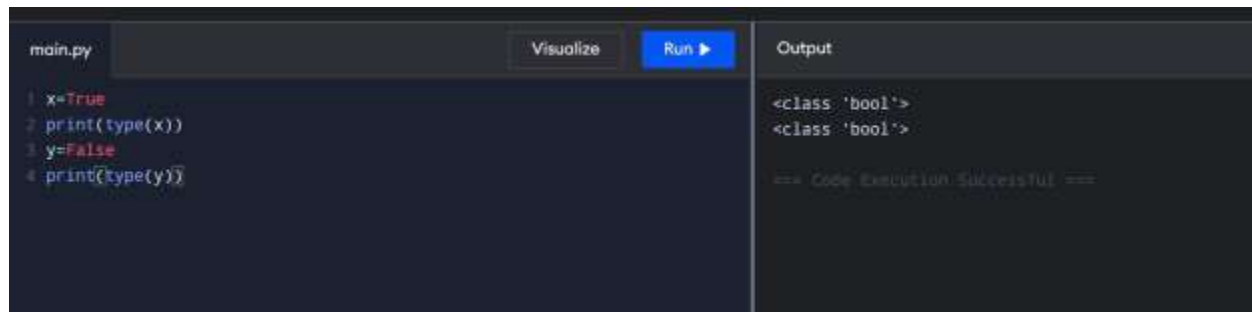
```
1 x=complex(1,2)
2 print(type(x))
3 print(x)
4 (1+2j)
5 z=1+2j
6 print(type(z))
7 z=1+2j
8 print(type(z))
9
```

The IDE has 'Visualize' and 'Run' buttons. The 'Output' pane on the right shows the following results:

```
<class 'complex'>
(1+2j)
<class 'complex'>
<class 'complex'>

=== Code Execution Successful ===
```

TASK #9



The screenshot shows a Python IDE with a file named 'main.py'. The code in the editor is as follows:

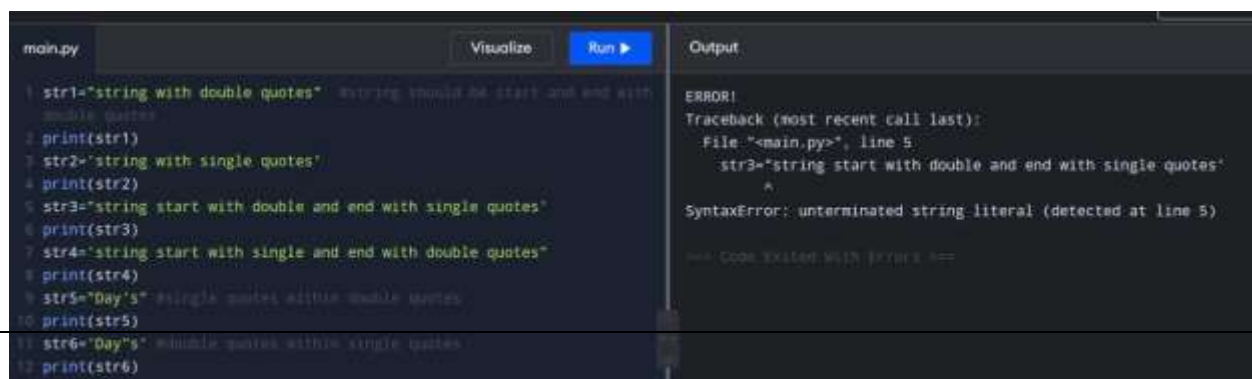
```
1 x=True
2 print(type(x))
3 y=False
4 print(type(y))
```

The IDE has 'Visualize' and 'Run' buttons. The 'Output' pane on the right shows the following results:

```
<class 'bool'>
<class 'bool'>

=== Code Execution Successful ===
```

TASK #10



The screenshot shows a Python IDE with a file named 'main.py'. The code in the editor is as follows:

```
1 str1="string with double quotes" #string should be start and end with double quotes
2 print(str1)
3 str2='string with single quotes'
4 print(str2)
5 str3="string start with double and end with single quotes"
6 print(str3)
7 str4='string start with single and end with double quotes'
8 print(str4)
9 str5="Day's" #single quotes within double quotes
10 print(str5)
11 str6="Day's" #double quotes within single quotes
12 print(str6)
```

The IDE has 'Visualize' and 'Run' buttons. The 'Output' pane on the right shows an error:

```
ERROR!
Traceback (most recent call last):
  File "<main.py>", line 5
    str3="string start with double and end with single quotes"
    ^
SyntaxError: unterminated string literal (detected at line 5)

=== Code Executed With Errors ===
```

TASK #11

main.py	Visualize	Run ▶	Output
<pre>1 print("This is a backslash (\\) mark.") 2 print("This is a tab \t key.") 3 print("These are \'single quotes\'") 4 print("These are \"double quotes\"") 5 print("This is a new line\nNew line.") 6</pre>			<pre>This is a backslash (\\) mark. This is a tab key. These are 'single quotes' These are "double quotes" This is a new line New line. === Code Execution Successful ===</pre>

TASK #12

main.py	Visualize	Run ▶	Output
<pre>1 string1="PYTHON TUTORIAL" 2 print(string1[0]) 3 print(string1[-15]) 4 print(string1[14]) 5 print(string1[-1]) 6 print(string1[4]) 7 print(string1[-11]) 8 print(string1[16]) # it gives error bcz 16 index doesn't exist</pre>			<pre>P Y A L O ERROR! N Traceback (most recent call last): File "<main.py>", line 8, in <module> IndexError: string index out of range === Code Executed With Errors ===</pre>

TASK #13

main.py	Visualize	Run ▶	Output
<pre>1 my_list1=[5,12,13,14] #the list contains all integer values 2 print(my_list1) 3 my_list2=['red','blue','black','white'] # the list contains all string values 4 print(my_list2) 5 my_list3=['red',12,112,12] #this list contains integer, string and float values 6 print(my_list3)</pre>			<pre>[5, 12, 13, 14] ['red', 'blue', 'black', 'white'] ['red', 12, 112, 12] === Code Execution Successful ===</pre>

TASK #14

main.py	Visualize	Run ▶	Output
<pre>1 my_list=[] 2 print(my_list)</pre>			<pre>[] === Code Execution Successful ===</pre>

TASK #15

main.py	Visualize	Run ▶	Output
<pre>1 color_list=["Red","Blue","Green","Black"] #the list contain 4 element #indices start at 0 and end at 3 2 print(color_list[0]) #return the 1st element 3 print(color_list[1],color_list[3]) 4 print(color_list[-1]) 5 print(color_list[4]) #gives error bec this index doesn't exist</pre>			<pre>Red Blue Black Black ERROR! Traceback (most recent call last): File "<main.py>", line 5, in <module> IndexError: list index out of range === Code Exited With Errors ===</pre>

TASK #16

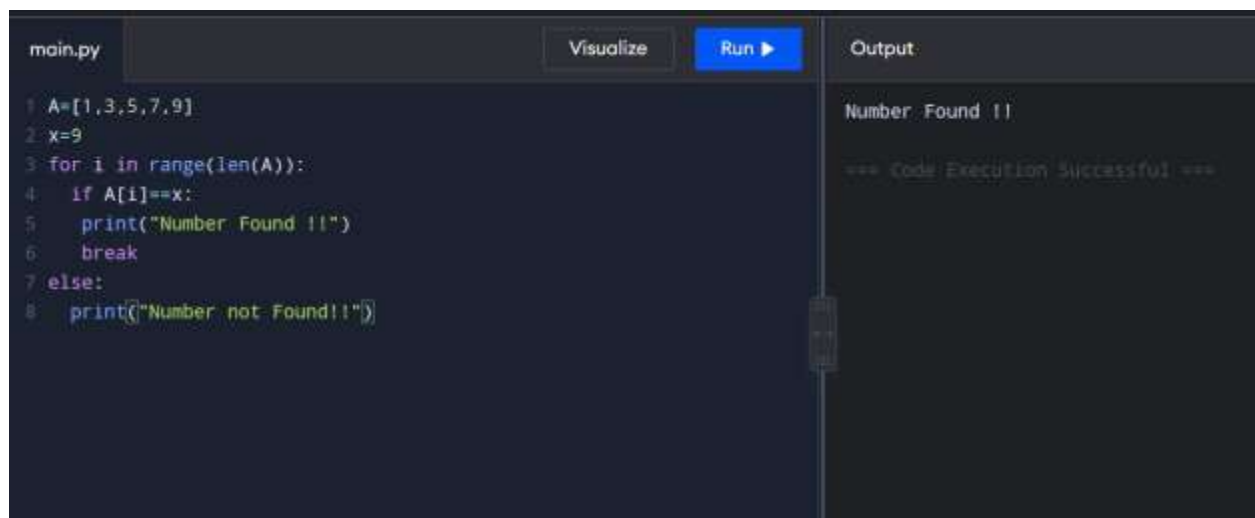
Programiz Python Online Compiler			
main.py	Visualize	Run ▶	Output
<pre>1 color_list1=["Red","Blue","Green","Black"] 2 print(color_list1[0:2]) #cut 1st two items 3 print(color_list1[1:2]) #cut 2nd item 4 print(color_list1[1:-2]) #cut 2nd item 5 print(color_list1[:3]) #cut 1st three items 6 print(color_list1[:]) # create copy of original list</pre>			<pre>['Red', 'Blue'] ['Blue'] ['Blue'] ['Red', 'Blue', 'Green'] ['Red', 'Blue', 'Green', 'Black'] === Code Execution Successful ===</pre>

ALGORITHMS

Algorithm no 01

Find if a number is present in array or not

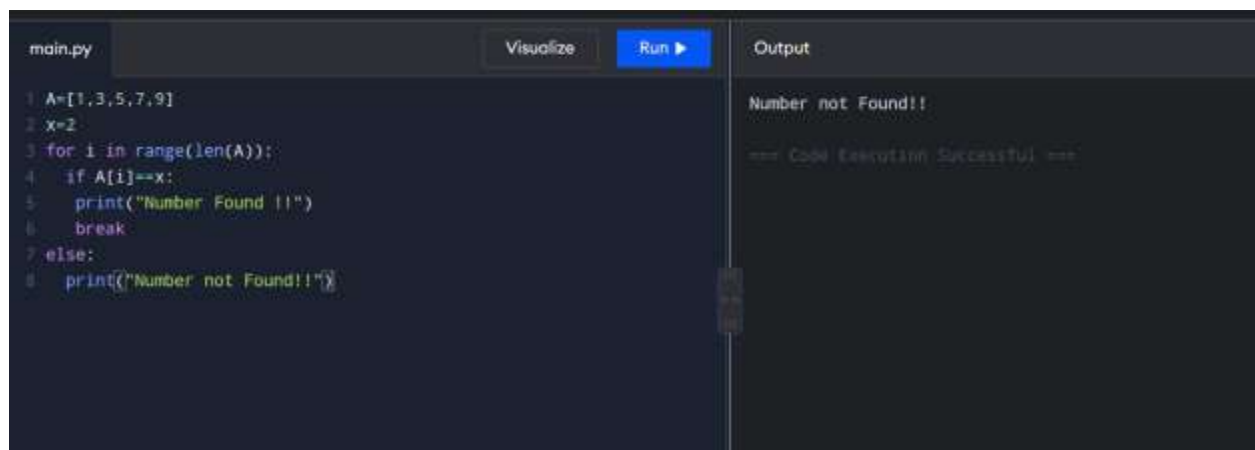
- If number is found



```
main.py Visualize Run ▶  
1 A=[1,3,5,7,9]  
2 x=9  
3 for i in range(len(A)):  
4     if A[i]==x:  
5         print("Number Found !!")  
6         break  
7 else:  
8     print("Number not Found!!")
```

Output
Number Found !!
=== Code Execution Successful ===

- If number is not found



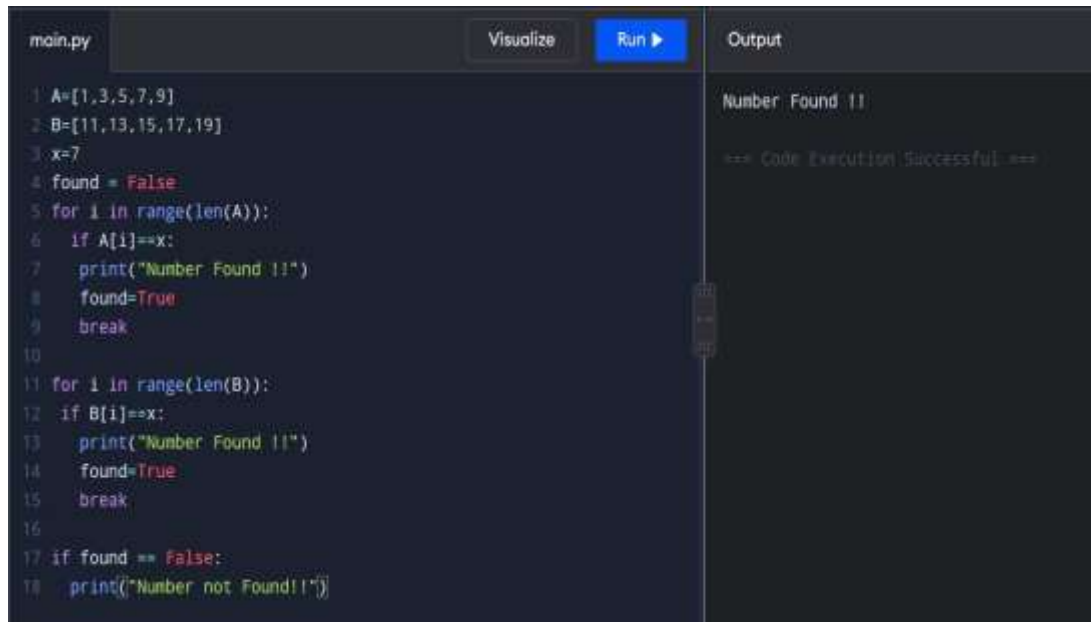
```
main.py Visualize Run ▶  
1 A=[1,3,5,7,9]  
2 x=2  
3 for i in range(len(A)):  
4     if A[i]==x:  
5         print("Number Found !!")  
6         break  
7 else:  
8     print("Number not Found!!")
```

Output
Number not Found!!
=== Code Execution Successful ===

Algorithm no 02

Find if a number is present in both arrays or not

- If number is found

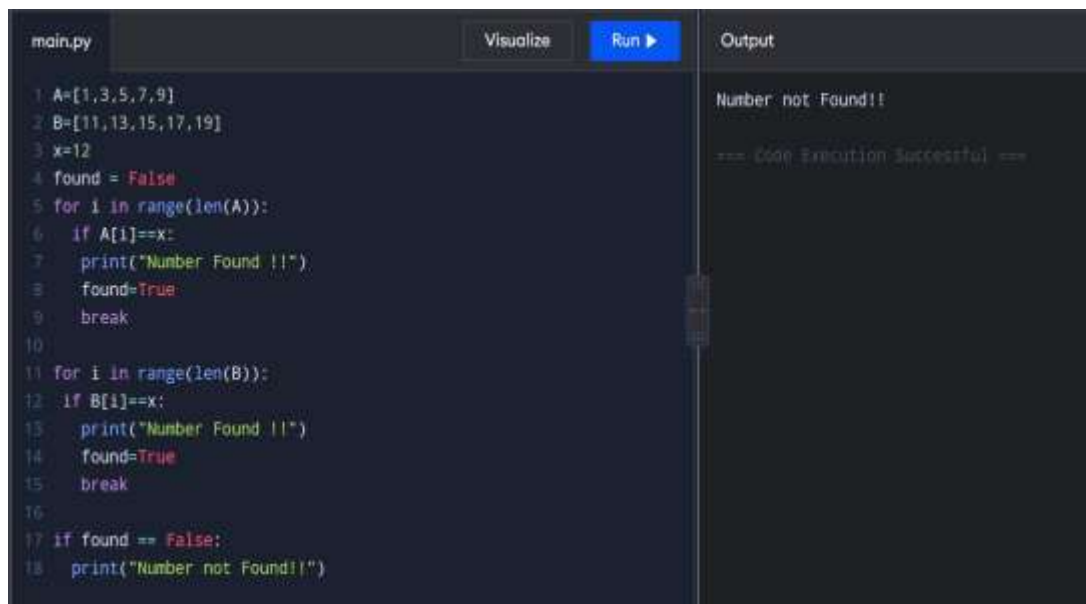


```
main.py Visualize Run ▶ Output
1 A=[1,3,5,7,9]
2 B=[11,13,15,17,19]
3 x=7
4 found = False
5 for i in range(len(A)):
6     if A[i]==x:
7         print("Number Found !!")
8         found=True
9         break
10
11 for i in range(len(B)):
12     if B[i]==x:
13         print("Number Found !!")
14         found=True
15         break
16
17 if found == False:
18     print("Number not Found!!")
```

Number Found !!

=== Code Execution Successful ===

- If number is not found



```
main.py Visualize Run ▶ Output
1 A=[1,3,5,7,9]
2 B=[11,13,15,17,19]
3 x=12
4 found = False
5 for i in range(len(A)):
6     if A[i]==x:
7         print("Number Found !!")
8         found=True
9         break
10
11 for i in range(len(B)):
12     if B[i]==x:
13         print("Number Found !!")
14         found=True
15         break
16
17 if found == False:
18     print("Number not Found!!")
```

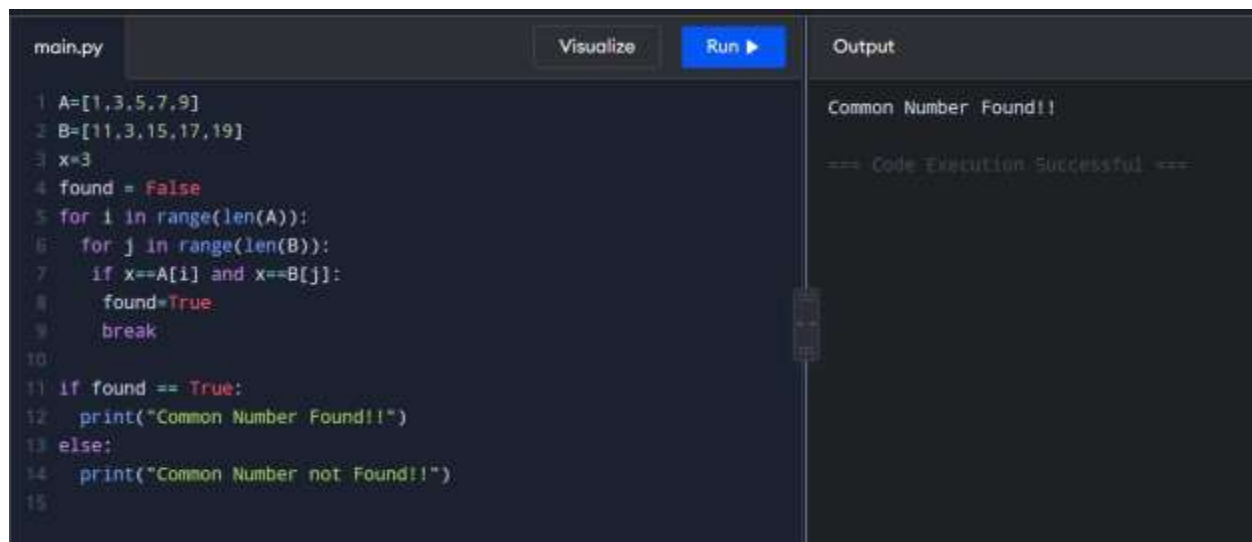
Number not Found!!

=== Code Execution Successful ===

Algorithm no 03

Find if a number is common in both arrays or not

- If common number is found

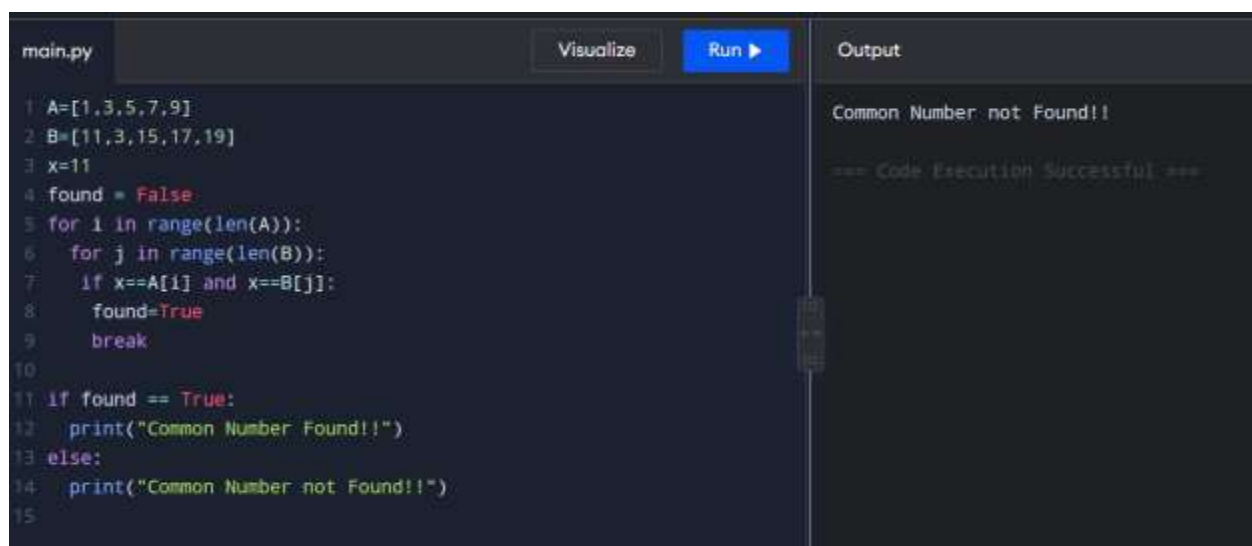


```
main.py Visualize Run ▶ Output
1 A=[1,3,5,7,9]
2 B=[11,3,15,17,19]
3 x=3
4 found = False
5 for i in range(len(A)):
6     for j in range(len(B)):
7         if x==A[i] and x==B[j]:
8             found=True
9             break
10
11 if found == True:
12     print("Common Number Found!!")
13 else:
14     print("Common Number not Found!!")
15
```

Common Number Found!!

=== Code Execution Successful ===

- If common number is not found



```
main.py Visualize Run ▶ Output
1 A=[1,3,5,7,9]
2 B=[11,3,15,17,19]
3 x=11
4 found = False
5 for i in range(len(A)):
6     for j in range(len(B)):
7         if x==A[i] and x==B[j]:
8             found=True
9             break
10
11 if found == True:
12     print("Common Number Found!!")
13 else:
14     print("Common Number not Found!!")
15
```

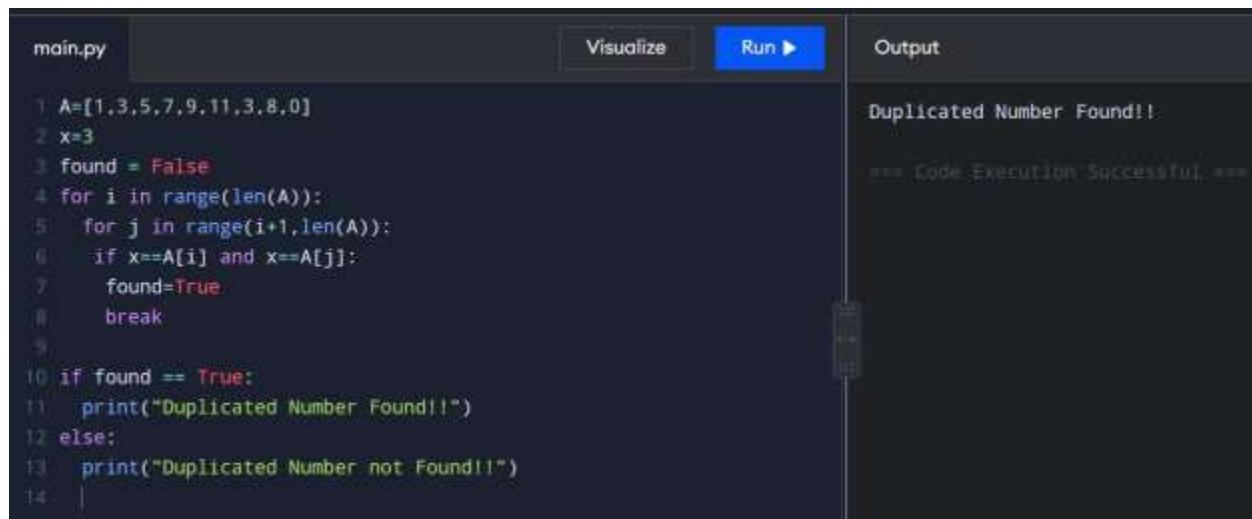
Common Number not Found!!

=== Code Execution Successful ===

Algorithm no 04

Find if a number is Duplicated in an array or not

- If a number is duplicated

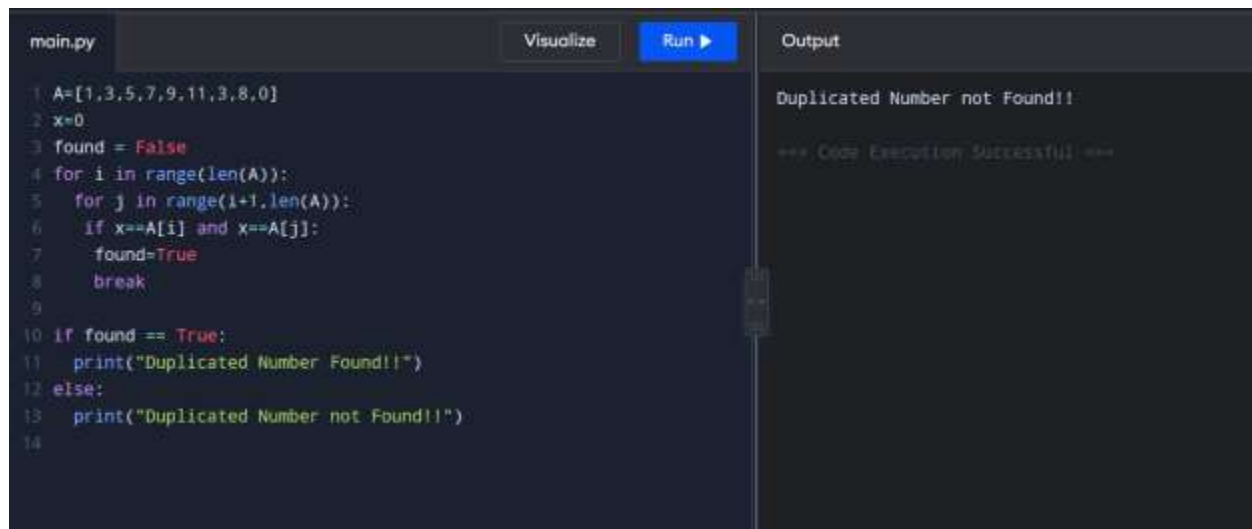


```
main.py Visualize Run ▶ Output
1 A=[1,3,5,7,9,11,3,8,0]
2 x=3
3 found = False
4 for i in range(len(A)):
5     for j in range(i+1,len(A)):
6         if x==A[i] and x==A[j]:
7             found=True
8             break
9
10 if found == True:
11     print("Duplicated Number Found!!")
12 else:
13     print("Duplicated Number not Found!!")
14 |
```

Duplicated Number Found!!

=== Code Execution Successful ===

- If a number is not duplicated



```
main.py Visualize Run ▶ Output
1 A=[1,3,5,7,9,11,3,8,0]
2 x=0
3 found = False
4 for i in range(len(A)):
5     for j in range(i+1,len(A)):
6         if x==A[i] and x==A[j]:
7             found=True
8             break
9
10 if found == True:
11     print("Duplicated Number Found!!")
12 else:
13     print("Duplicated Number not Found!!")
14 |
```

Duplicated Number not Found!!

=== Code Execution Successful ===